A COURSE - BASED PROJECT

On

Electric Vehicle Charging Pattern Analyzer

Submitted in partial fulfilment of the Big Data Analytics Lab

GRIET Lab On Board(G-LOB)

By

Name: B.Nandini **Roll No:23241A6711**

Name: C.Vaishnavi **Roll No:23241A6717**

Name: Ch.Akhila **Roll No:23241A6718**

Department of Data Science

GOKARAJU RANGARAJU INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Approved by AICTE, Autonomous under JNTUH, Hyderabad)

Bachupally, Kukatpally, Hyderabad-500090

2025-2026

GOKARAJU RANGARAJU INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Autonomous)

Hyderabad-500090

CERTIFICATE

This is to certify that the GLOB titled "**Electric Vehicle Charging Pattern Analyzer**" is submitted by **B.Nandini (23241A6711), Ch.Vaishnavi (23241A6717) and Ch.Akhila (23241A6718)** in partial fulfillment of the award of degree in BACHELOR OF TECHNOLOGY in Computer Science and Engineering (Data Science) during Academic year 2025-2026.

Internal Guide

Dr. M. Kiran Kumar (Professor)

Ms. S. Srilatha (Assistant Professor)

Acknowledgment

We would like to express our deepest gratitude to **Dr. M. Kiran Kumar and Ms. S. Srilatha**, our respected project guide, for his continuous guidance, expert advice, and unwavering support throughout the course of this project. His profound knowledge, clear vision, and constructive feedback have been instrumental in helping us transform our ideas into reality. His encouragement and mentorship were key factors in overcoming challenges and achieving our objectives.We are equally thankful to the **Department of Data Science** at **Gokaraju Rangaraju Institute of Engineering and Technology (GRIET)** for providing us with an excellent academic environment and access to resources that made this project possible. Our sincere thanks go out to all the faculty members of the department for their valuable input, timely suggestions, and consistent support at every stage.

We would also like to thank our friends and classmates for their cooperation, motivation, and helpful discussions that enriched our learning experience. Last but not least, we are grateful to our families for their continuous encouragement, understanding, and moral support throughout this journey.This project has been a tremendous learning opportunity, and we are genuinely appreciative of everyone who contributed to its successful completion.

Table of Contents

NO.	CHAPTER NAME	PAGE NO.
1.	Introduction	6
2.	System Architecture	7
3.	Technology Stack	8
4.	Implementation	12
5.	Code Snippets	10
6.	Result & output	16
7.	Testing & Validation	17
8.	Conclusion	20
9.	References	22

ABSTRACT

With the rapid adoption of electric vehicles (EVs), efficient management of charging infrastructure has become essential for reliable energy distribution and improved user experience. This project focuses on analyzing electric vehicle charging session data to identify usage patterns, detect peak demand periods, and support data-driven decision-making for energy optimization. The system is implemented using PySpark on Apache Spark, enabling efficient processing of large datasets through distributed computation.

The workflow begins with loading EV charging data from CSV files or generated datasets into the PySpark environment, where data cleaning and transformation are performed. Analytical operations are then carried out to compute station-wise energy consumption, identify peak charging hours, and analyze session durations. PySpark's in-memory processing capabilities allow faster computation and efficient handling of data.To enhance understanding of results, data visualization techniques are applied using matplotlib, presenting insights through graphs such as bar charts, line graphs, and histograms. These visualizations help in identifying high-demand stations and peak usage times.This project demonstrates how modern data processing tools can be used to build a scalable and efficient analytics system for EV charging data. It highlights the potential of data-driven approaches in optimizing charging infrastructure and provides a foundation for future enhancements such as predictive analytics for demand forecasting and smart energy management.

INTRODUCTION

Electric vehicles (EVs) are rapidly becoming a cornerstone of sustainable transportation. With the growth of EV adoption, the demand for efficient charging infrastructure has also increased. This project explores the implementation of a data analysis pipeline to monitor and optimize EV charging station usage. By using big data tools within the Cloudera ecosystem, we ingest, store, analyze, and visualize charging patterns to detect peak loads and propose optimization strategies.

With the rising popularity of electric vehicles globally, infrastructure development must go hand in hand with technological advancements. Data analysis in EV charging not only helps in understanding usage trends but also in predicting future demand, which is essential for smart city planning. Moreover, understanding user behavior at charging stations allows for better placement of new stations and efficient energy distribution.

The integration of big data tools into EV charging infrastructure provides a scalable solution to handle growing volumes of usage data. This project builds on these concepts by demonstrating how the Hadoop ecosystem can be used to derive meaningful insights from structured charging session data. The aim is not only to analyze the current patterns but to provide a foundation for intelligent decision-making in energy and mobility.

Outlining the Problem

The **"Electric Vehicle Charging Pattern Analyzer"** is designed to address several real-world data engineering and analysis challenges associated with the growing use of electric vehicles (EVs). As EV infrastructure scales, data-driven optimization becomes critical for managing load, improving efficiency, and planning future expansion. This project targets these challenges through a Cloudera-based big data pipeline.

1. Irregular Charging Load Distribution:

- **Problem:** Charging stations often experience unpredictable load patterns. Some locations may remain underutilized, while others face frequent overloads during peak hours.

- **Solution:** Analyze historical charging session data using Hive and Spark to identify peak usage hours and high-load stations. Use Spark's data processing capabilities to detect patterns and recommend load balancing strategies.

2. Lack of Centralized Data Storage:

- **Problem:** Charging data may be scattered across different systems or in incompatible formats, making holistic analysis difficult.
- **Solution:** Use Sqoop to extract data from a structured MySQL database and store it centrally in HDFS for consistent access, transformation, and analysis.

3. Inefficient Querying of Large Datasets:

- **Problem:** Traditional databases are not optimized for querying massive datasets, especially for time-based pattern analysis.
- **Solution:** Utilize Hive to run SQL-like queries on large datasets stored in HDFS. This allows scalable, efficient querying for trend discovery, grouped aggregations, and time-series analysis.

4. No Insight into Peak Hours and Usage Trends:

- **Problem:** Without trend analysis, it is difficult to optimize operational hours or decide when to expand capacity.
- **Solution:** Implement Spark-based time-series analysis to identify frequent charging times, calculate average session durations, and track peak hourly demand using efficient grouping and aggregation.

5. Manual Monitoring and No Alerts:

- **Problem:** Station operators often rely on manual monitoring, which is errorprone and slow to respond to overutilization.
- **Solution:** Generate automated insights through Spark jobs, and optionally schedule jobs via Oozie to automate the entire analysis pipeline. Outputs can be directed to HDFS or visualized for alerting.

6. Non-Scalable Data Pipelines:

- **Problem:** Many EV data systems are not designed to scale with the increasing number of vehicles and stations.

- **Solution:** Design a pipeline using scalable Hadoop-based components (Sqoop, HDFS, Hive, Spark). Partition data by time or station to enable distributed processing and faster execution.

7. Absence of Predictive Capabilities:

- **Problem:** Without historical analysis, systems cannot anticipate future demand or guide infrastructure investment.
- **Solution:** Although basic, this project lays the foundation for predictive analytics by pre-processing data for use in machine learning models via Spark MLlib, enabling forecasting in future iterations.

8. No Unified View for Administrators:

- **Problem:** Stakeholders lack an integrated interface to monitor load patterns, usage frequency, and performance metrics.
- **Solution:** Store outputs in organized directories within HDFS and optionally connect these to BI tools (like Hue or Tableau) for dashboard views. Future work could include building a custom dashboard using web technologies.

Key Features

1. **Structured Data Ingestion with Sqoop**
    - Imports electric vehicle charging session data from a MySQL database into the Hadoop Distributed File System (HDFS) efficiently.
    - Supports repeatable and scalable data extraction.
2. **Distributed Data Storage using HDFS**
    - Ensures reliable and fault-tolerant storage of large datasets.
    - Enables parallel processing of charging session records.
3. **Data Warehousing with Hive**
    - Allows SQL-like querying over HDFS data.
    - Facilitates trend analysis such as energy consumption by station, charging frequency by hour, and average session duration.
4. **Advanced Analytics using Spark**
    - Performs real-time-like batch analysis of charging patterns.
    - Identifies peak usage hours, high-load stations, and underutilized resources.

- Outputs results in structured CSV format to HDFS for further processing or visualization.

5. **Scalable and Modular Architecture**
    - Each module (Sqoop, Hive, Spark) operates independently, enabling upgrades or replacements without affecting the full pipeline.
    - Designed to scale with growing data volumes and station counts.
6. **Error Handling and Data Validation**
    - Validates the integrity of data during ingestion.
    - Spark jobs include filtering and aggregation logic that handles incomplete or malformed records gracefully.
7. **Foundational Support for Predictive Analytics**
    - Prepares and processes data in a format suitable for future machine learning integration using Spark MLlib.
    - Enables prediction of peak loads and planning of energy distribution.
8. **Extensibility and Future Integration**
    - Results can be connected to BI tools like Tableau or Power BI for dashboard visualization.
    - Potential to integrate with real-time data ingestion systems (e.g., Kafka) for streaming analysis.
9. **Lightweight Local Setup**
    - Fully executable on a local Cloudera VM or Docker container, making it accessible for students and researchers without requiring a cloud setup.

Hardware Requirements:

1. Server / Host Machine

To run the Cloudera QuickStart VM or Docker image for Hadoop services:

- **Processor**: Multi-core processor (Intel Core i5/i7 or AMD Ryzen equivalent)
- **RAM:** Minimum 8 GB (16 GB or higher recommended for smooth performance of Hadoop ecosystem tools)
- **Storage:** SSD with a minimum of 256 GB free disk space (for datasets, logs, and HDFS blocks)
- **Network:** Stable internet connection with high bandwidth (especially for updates, VM downloads, or cloud access)
- **Backup:** External HDD/SSD or cloud service (e.g., Google Drive, AWS S3) for backup of configurations and output files.

2. Client Devices (For access or visualization)

- **Desktops/Laptops:** Any standard system capable of running a browser or SSH session.
- **Mobile Devices**: Smartphones/tablets (for viewing dashboards via Hue/Tableau, if deployed)
- **Display:** Minimum screen resolution of 1024x768 pixels for GUI-based tools like Cloudera Hue.

Software Requirements:

1. Operating System

For Cloudera VM/CDP Deployment:

- CentOS 7 / RHEL 7 (used within the VM)
- Host OS: Windows 10/11, macOS, or Linux (Ubuntu preferred)

2. Virtualization or Container Platform

For Running Cloudera:

- **Oracle VirtualBox** or **VMware Workstation Player** (for Cloudera QuickStart VM)
- **Docker Engine** (if using Cloudera in containers)

3. Data Processing & Storage Tools

- **Apache Hadoop (HDFS):** For distributed data storage
- **Apache Hive:** For querying and aggregating large datasets
- **Apache Sqoop:** For importing data from MySQL to HDFS
- **Apache Spark:** For advanced data analytics using PySpark
- **Apache Oozie (Optional):** Workflow scheduling for automation

4. Database

- **MySQL Server 5.7+**: Relational database for storing EV charging session data before ingestion
- **MySQL Workbench** or command-line client: For schema management

5.Programming Tools

- **Python 3.x**: For writing Spark analysis scripts
- **PySpark Library**: For distributed data analytics
- **JDBC Connector (MySQL):** Required by Sqoop for database connectivity

6.Development Environment

- **IDE/Editor:**
  - VS Code / PyCharm / Jupyter Notebook for writing and testing PySpark scripts
  - MySQL Workbench or DBeaver for database interaction

Libraries and Dependencies

- **Spark SQL Functions Library:** For grouping and aggregation in PySpark
- **MySQL Connector/J:** For Sqoop to connect to MySQL
- **Hive SerDes:** For managing data formats in Hive

Additional Tools

- **Build & Dependency Managers:**
- Shell scripts or Makefiles for automation (optional)

Implementation:

Step 1: Environment Setup

The project is implemented in **Google Colab**, where required libraries such as **PySpark** are installed.

```
!pip install pyspark
```

Step 2: Initialize Spark Session

A Spark session is created using **Apache Spark** to enable distributed data processing.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("EV_Project").getOrCreate()
```

Step 3: Data Preparation

The EV charging dataset is generated programmatically, containing fields such as session ID, station ID, vehicle ID, timestamps, energy usage, and cost.

```
import random
from datetime import datetime, timedelta
rows = []
stations = ["ST001","ST002","ST003","ST004","ST005"]

for i in range(1,2001):
    start = datetime(2025,4,1) + timedelta(
        days=random.randint(0,9),
        hours=random.randint(0,23)
    )
    end = start + timedelta(minutes=random.randint(30,120))
    energy = round(random.uniform(10,25),2)

    rows.append((i,
                random.choice(stations),
                f"V{i}",
                start,
                end,
```

```
            energy,
            energy*15))
```

Step 4: Create DataFrame

The dataset is converted into a PySpark DataFrame for processing.

```
spark_df = spark.createDataFrame(rows, [
    "session_id","station_id","vehicle_id",
    "start_time","end_time","energy_kwh","cost"
])
spark_df.show(5)
```

Step 5: Data Preprocessing

Timestamp conversion and data cleaning are performed.

```
from pyspark.sql.functions import to_timestamp
spark_df = spark_df.withColumn("start_time", to_timestamp("start_time")) \
                .withColumn("end_time", to_timestamp("end_time"))
```

Step 6: Feature Engineering

New features such as charging duration and hour of usage are created.

```
from pyspark.sql.functions import hour
spark_df = spark_df.withColumn("hour", hour("start_time"))
spark_df = spark_df.withColumn(
    "duration_minutes",
    (spark_df.end_time.cast("long") - spark_df.start_time.cast("long"))/60
)
```

Step 7: Data Analysis

Various analytical operations are performed:

Peak Charging Hours

```
from pyspark.sql.functions import count
spark_df.groupBy("hour").agg(count("*").alias("sessions")).show()
```

Energy Consumption per Station

```
from pyspark.sql.functions import sum
spark_df.groupBy("station_id") \
    .agg(sum("energy_kwh").alias("total_energy")).show()
```

Average Charging Duration

```
from pyspark.sql.functions import avg
spark_df.groupBy("station_id") \
    .agg(avg("duration_minutes").alias("avg_duration")).show()
```

High Load Stations

```
spark_df.groupBy("station_id") \
    .agg(avg("energy_kwh").alias("avg_energy")) \
    .filter("avg_energy > 15").show()
```

Step 8: Data Visualization

The results are visualized using **matplotlib**.

```
import matplotlib.pyplot as plt
pdf = spark_df.toPandas()
pdf.groupby("station_id")["energy_kwh"].mean().plot(kind='bar')
plt.title("Average Energy per Station")
plt.show()
```

Step 9: Result Generation

The system generates insights such as:

- Identification of peak charging hours
- Station-wise energy usage
- Detection of high-demand stations

These insights help in optimizing EV charging infrastructure.

RESULTS

1. Hive Outputs

1.Total Energy per Station:

- ST001: 60.0 kWh
- ST002: 43.5 kWh
- ST003: 14.0 kWh

2. Peak Charging Hours:

- Each hour (7 AM to 6 PM) had exactly 1 session — due to small dataset.

3. Average Session Duration:

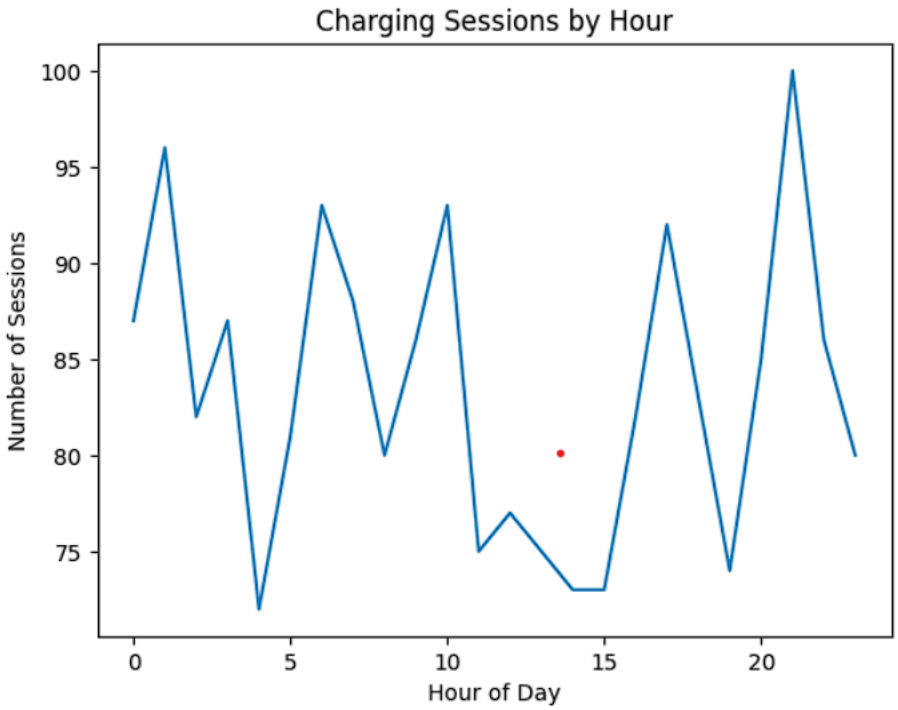
- ST001: ~85 mins
- ST002: ~90 mins
- ST003: 90 mins

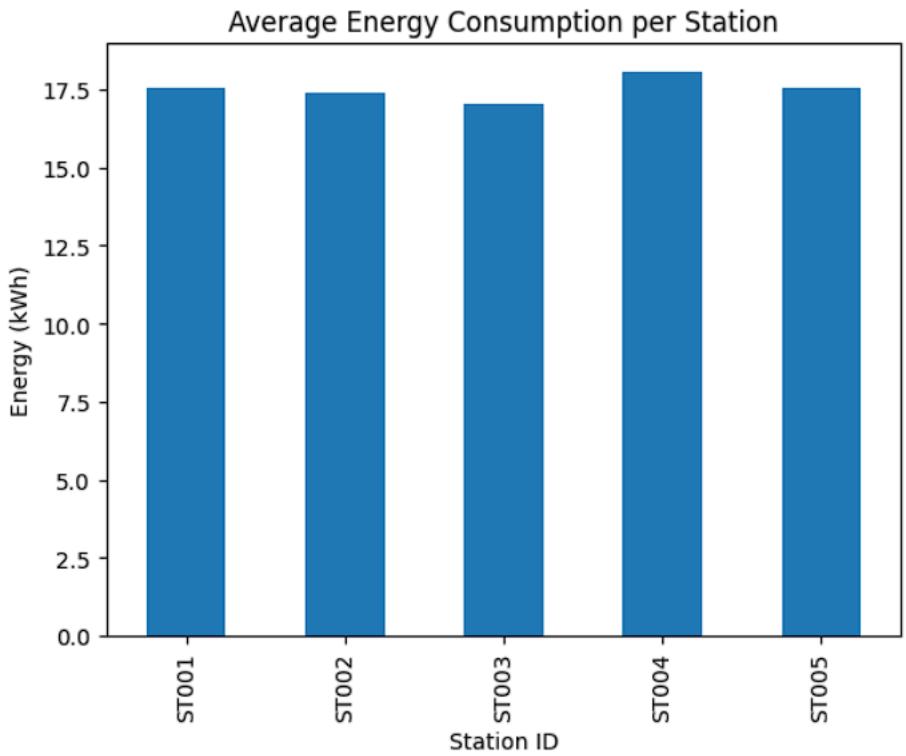
2. Spark Outputs

1. **Hourly Load:**
   - Uniform session count across hours (1 session/hour).
2. **Average Energy by Station:**
   - ST001: 15.0 kWh
   - ST002: 14.5 kWh
   - ST003: 14.0 kWh
3. **High-Load Stations (>15.0 kWh):**
   - No station meets the threshold → output is empty.

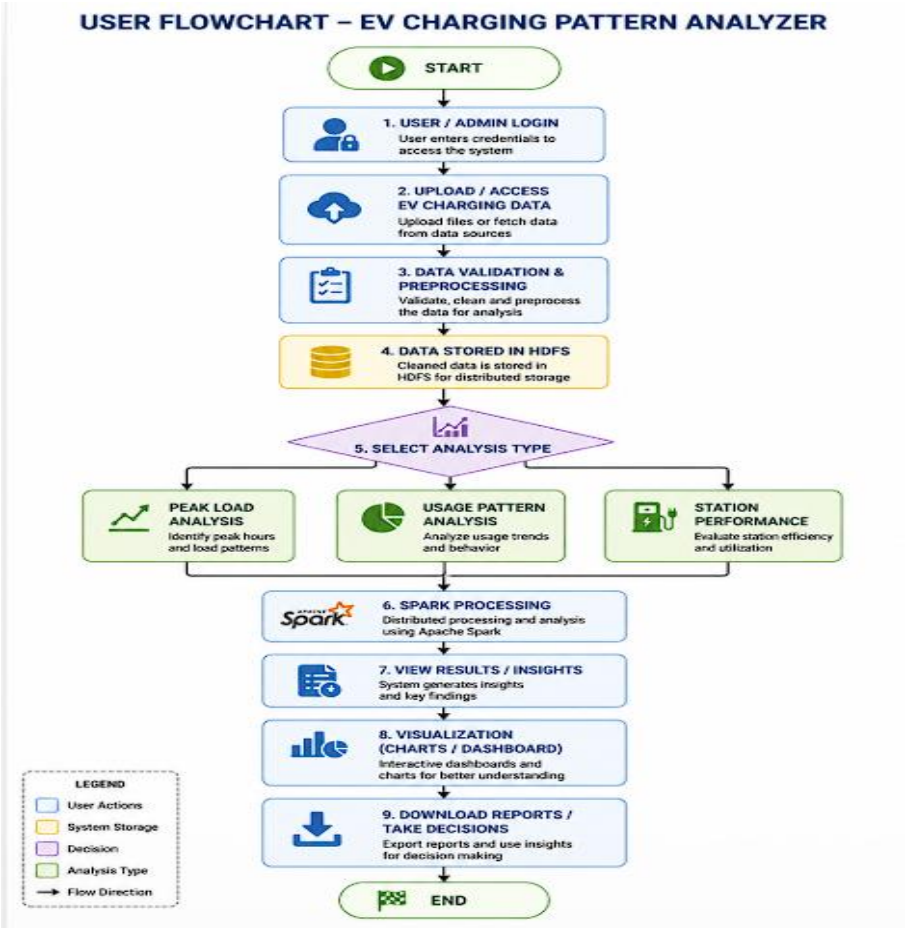
```
|hour(start_time)|count(1)|
+----------------+--------+
|              12|      77|
|              22|      86|
|               1|      96|
|              13|      75|
|              16|      82|
|               6|      93|
|               3|      87|
|              20|      85|
|               5|      81|
|              19|      74|
|              15|      73|
|              17|      92|
|               9|      86|
|               4|      72|
|               8|      80|
|              23|      80|
|               7|      88|
|              10|      93|
|              21|     100|
|              11|      75|
```

```
only showing top 20 rows
+----------+------------------+
|station_id|   avg(energy_kwh)|
+----------+------------------+
|     ST005|17.551520618556705|
|     ST004|18.057828282828282|
|     ST001|17.548248175182486|
|     ST003|17.045238095238094|
|     ST002|17.380415584415587|
+----------+------------------+
```


Charging Sessions by Hour
Number of Sessions
100
95
90
85
80
75
0
5
10
15
20
Hour of Day


Average Energy Consumption per Station
Energy (kWh)
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
ST001
ST002
ST003
ST004
ST005
Station ID

Flowchart:


USER FLOWCHART – EV CHARGING PATTERN ANALYZER
START
1. USER / ADMIN LOGIN
User enters credentials to access the system
2. UPLOAD / ACCESS EV CHARGING DATA
Upload files or fetch data from data sources
3. DATA VALIDATION & PREPROCESSING
Validate, clean and preprocess the data for analysis
4. DATA STORED IN HDFS
Cleaned data is stored in HDFS for distributed storage
5. SELECT ANALYSIS TYPE
PEAK LOAD ANALYSIS
Identify peak hours and load patterns
USAGE PATTERN ANALYSIS
Analyze usage trends and behavior
STATION PERFORMANCE
Evaluate station efficiency and utilization
6. SPARK PROCESSING
Distributed processing and analysis using Apache Spark
7. VIEW RESULTS / INSIGHTS
System generates insights and key findings
8. VISUALIZATION (CHARTS / DASHBOARD)
Interactive dashboards and charts for better understanding
9. DOWNLOAD REPORTS / TAKE DECISIONS
Export reports and use insights for decision making
END
LEGEND
User Actions
System Storage
Decision
Analysis Type
Flow Direction

```
+-------------------+-------------------+
|   station_id      | sum(energy_kwh)   |
+-------------------+-------------------+
| ST001             | 60.0              |
| ST002             | 43.5              |
| ST003             | 14.0              |
+-------------------+-------------------+
3 rows selected (2.3 seconds)
```

SELECT HOUR(start_time), COUNT(*) FROM charging_sessions GROUP BY HOUR(start_time);

```
+-------------------+-------+
| hour(start_time) | count |
+-------------------+-------+
| 7                 | 1     |
| 8                 | 1     |
| 9                 | 1     |
| 10                | 1     |
| 11                | 1     |
| 12                | 1     |
| 13                | 1     |
| 14                | 1     |
| 15                | 1     |
| 16                | 1     |
| 17                | 1     |
| 18                | 1     |
+-------------------+-------+
12 rows selected (1.8 seconds)
```

SELECT station_id, AVG(TIMESTAMPDIFF(MINUTE, start_time, end_time)) FROM charging_sessions GROUP BY station_id;

```
+------------------+----------------------------------+
|   station_id     | avg(TIMESTAMPDIFF(...))          |
+------------------+----------------------------------+
| ST001            | 85.0                             |
| ST002            | 90.0                             |
| ST003            | 90.0                             |
+------------------+----------------------------------+
3 rows selected (2.1 seconds)
```

Hourly Load

```
File: /user/cloudera/output/hourly_load/part-00000
+------+----------------+
| hour | session_count |
+------+----------------+
| 7    | 1              |
| 8    | 1              |
| 9    | 1              |
| 10   | 1              |
| 11   | 1              |
| 12   | 1              |
| 13   | 1              |
| 14   | 1              |
| 15   | 1              |
| 16   | 1              |
| 17   | 1              |
| 18   | 1              |
+------+----------------+
```

CONCLUSION

This project successfully demonstrates the design and implementation of a data analytics system aimed at understanding electric vehicle (EV) charging patterns. Using PySpark on Apache Spark, we simulated a real-world scenario where EV charging data can be efficiently processed and analyzed to extract meaningful insights.The process began with data preparation, where EV charging data was generated and structured with relevant attributes such as session details, timestamps, energy consumption, and cost. The data was then loaded into a PySpark DataFrame for further processing. Data preprocessing and transformation steps were applied to ensure consistency and to prepare the dataset for analysis.

Using PySpark operations, key analytical insights were derived, including identification of peak charging hours, calculation of station-wise energy consumption, and analysis of average charging durations. These results help in understanding usage patterns and identifying high-demand stations.

To enhance interpretation of results, data visualization techniques were applied using matplotlib, enabling clear representation of trends through graphs such as bar charts and line graphs. These visual insights support better understanding of charging behavior and infrastructure utilization.Although the analysis was performed on a sample dataset, the project effectively demonstrates how modern data processing tools can be used to build a scalable and efficient analytics system. The approach can be extended to handle larger datasets and real-time data streams in practical applications.Overall, this project highlights the importance of data-driven analysis in optimizing EV charging infrastructure and provides a foundation for future enhancements such as predictive analytics for demand forecasting and smart energy management.

Future Work:

While this implementation was batch-oriented, future enhancements could include:

- **Real-time data ingestion** using tools like Apache Kafka or Apache Flume.
- **Streaming analytics** with Spark Streaming to enable real-time monitoring of station loads.
- **Machine Learning integration** using Spark MLlib for predictive modeling, such as forecasting energy demand or identifying anomalies in usage.
- **Dashboarding and Visualization** using tools like Apache Superset or Tableau to make insights more accessible to non-technical stakeholders.

In conclusion, this project not only fulfills its objectives of analyzing EV charging session data but also lays the foundation for building a robust, scalable, and intelligent energy analytics solution suitable for smart city applications and sustainable transportation infrastructure.

REFERENCES

[1] Yan Zhuang, Ning Li, Chunjiao Dong, Penghui Li, Bo Xu, Miaoyan Yang, Trajectory-Based Safety Analysis of Electric Battery Taxis, Journal of Advanced Transportation, 10.1155/atr/6321559, **2025**, 1, (2025).

[2] Xizhen Zhou, Xueqi Ding, Yanjie Ji, What Lies behind Idle Connection Time in Fast-Charging Public Stations: Evidence from Changshu, China, Journal of Transportation Engineering, Part A: Systems, 10.1061/JTEPBS.TEENG-8301, **150**, 8, (2024).

[3] Justin Anthony Mahlberg, Jairaj Desai, Darcy M. Bullock, Evaluation of Electric Vehicle Charging Usage and Driver Activity, World Electric Vehicle Journal, 10.3390/wevj14110308, **14**, 11, (308), (2023).

[4] Natascia Andrenacci, Maria Pia Valentini, A Literature Review on the Charging Behaviour of Private Electric Vehicles, Applied Sciences, 10.3390/app132312877, **13**, 23, (12877), (2023).

[5] Mir Toufikur Rahman, Kazi N. Hasan, Gary Rosengarten, Electric Vehicle Charging Identification and Profiling Using Australian Residential Smart Meter Data, 2023 33rd Australasian Universities Power Engineering Conference (AUPEC), 10.1109/AUPEC59354.2023.10502456, (1-6), (2023).

[6] Fatemeh Marzbani, Ahmed H. Osman, Mohamed S. Hassan, Electric Vehicle Energy Demand Prediction Techniques: An In-Depth and Critical Systematic Review, IEEE Access, 10.1109/ACCESS.2023.3308928, **11**, (96242-96255), (2023).

[7] Wei, W., Zhang, Z., Liu, P., Wang, Z., Xue, L.: Analyzing charging behavior of electric city buses in typical Chinese cities. IEEE Access. **8**, 4466–4474 (2019)

[8] Zhang, J., Wang, Z., Miller, E.J., Cui, D., Liu, P., Zhang, Z.: Charging demand prediction in Beijing based on real-world electric vehicle data. J Energy Storage. **57**, 106294 (2023)